

Complete Backend Flow — Line-by-Line Reference

This document is the authoritative developer guide for the entire backend. It covers every source file, every database table and field, every seed operation, and every Docker configuration — line by line.

See also:

- [API SPECIFICATION.md](#) — concise endpoint contracts (request/response shapes)
- [DATABASE SCHEMA.md](#) — ER diagram and migration history
- [BACKEND SERVICES.md](#) — service/utility method reference table
- [NEXT ROUTE FLOW.md](#) — deep trace of route deduplication algorithm
- [DOCKER SETUP.md](#) — Docker onboarding and troubleshooting

Table of Contents

1. [Folder Structure](#)
2. [Server Entry Point — index.ts](#)
3. [Express App — app.ts](#)
4. [JWT Middleware — middleware.ts](#)
5. [Type Definitions — api.interfaces.ts and route.dto.ts](#)
6. [API Handlers — api.ts](#)
7. [API Utilities — api.utilities.ts and reshape.Utilites.ts](#)
8. [Service Guards — CheckSession and playerIdTypeCheck](#)
9. [Game Service — game.service.ts](#)
10. [Route Service — route.service.ts](#)
11. [Prisma Client — prisma.ts](#)
12. [DB Operation: getOrCreatePlayerSession](#)
13. [DB Operation: HasSessionService](#)
14. [DB Operation: getActiveRouteForPlayerSession](#)
15. [DB Operation: LoadRouteDTOService](#)
16. [DB Operation: GetPolinatorNames](#)
17. [DB Operation: getNextRouteForPlayerSession \(Transaction\)](#)
18. [DB Operation: addChildrenAdultsService](#)
19. [Portal DB Operations \(Read\)](#)
20. [Database Schema — schema.prisma](#)
21. [Seed Data — seed.js / seed.pg.js](#)
22. [Docker Configurations](#)
23. [Nginx Reverse Proxy](#)
24. [Request Lifecycle — End to End](#)

1. Folder Structure

```
backend/
├─ Dockerfile                # Production build – MySQL/MariaDB
├─ Dockerfile.pg             # Production build – PostgreSQL
├─ dockerfile.dev            # Development image (shared by MySQL + PG dev stacks)
├─ prisma.config.ts          # Prisma config pointing at prisma/schema.prisma (MySQL)
├─ prisma.config.pg.ts       # Prisma config pointing at prisma/postgresql/schema.prisma (PG)
├─ package.json
├─ tsconfig.json
├─ vitest.config.ts
├─
├─ prisma/
│  └─ schema.prisma          # MySQL/MariaDB Prisma schema
│  └─ seed.js                # MySQL seed script
│  └─ seed.pg.js             # PostgreSQL seed script (identical content, PG string escaping)
│  └─ migrations/            # MySQL migration history
│     └─ postgresql/
│        └─ schema.prisma    # PostgreSQL Prisma schema
```

```

└─ migrations/          # PostgreSQL migration history
└─ generated/           # Auto-generated Prisma client (gitignored)
└─ JsonDataBackups/     # Static JSON snapshots of route data (reference only, not imported)
└─ src/
  └─ index.ts
  └─ app.ts
  └─ Back-end_types/
    └─ route.dto.ts
  └─ middleware/
    └─ middleware.ts
  └─ api/
    └─ api.ts
    └─ interfaces/
      └─ api.interfaces.ts
    └─ Utilities/
      └─ api.utilities.ts
      └─ reshape.Utilites.ts  ← typo in filename is intentional (historical)
  └─ services/
    └─ game.service.ts
    └─ route.service.ts
    └─ gameServiceUtils/
      └─ CheckSession.service.ts
      └─ PlayeridTypeCheck.ts
    └─ db/
      └─ prisma.ts
      └─ DatabaseOperationsServices/
        └─ Create/
          └─ getOrCreatePlayerSession.service.ts
        └─ Read/
          └─ hassession.service.ts
          └─ getActiveRouteForPlayerSession.service.ts
          └─ LoadRouteDTO.service.ts
          └─ GetpollinatorNames.ts
          └─ getChildrenandAdultsbySession.ts
          └─ getSessionsandPlayeridsbyChildSize.ts
          └─ getsessionidsandplayeridsbyfamilysize.ts
          └─ getsessionandPlayeridbyAdultsize.ts
          └─ getchildadultdatabystartandend.ts
          └─ getchildnumandadulttoenddate.ts
          └─ getchildnumandadultdatetoforever.ts
          └─ getTotalRoutesCompleted.ts          # route-stats 4.1 session
          └─ getTotalConnectedUsers.ts          # route-stats 4.2 session
          └─ getUsersStartedMoreThanOneRoute.ts # route-stats 4.3 session
          └─ getUsersCompletedMoreThanOneRoute.ts # route-stats 4.4 session
          └─ getUsersNeverStartedRoute.ts       # route-stats 4.5 session
          └─ getuserswhostartedbutdidnotfinishanyroute.ts # route-stats 4.6 session
          └─ getTotalReplays.ts                 # route-stats 4.7 session
          └─ getAvgRoutesPerUserBySession.ts    # route-stats 4.8 session
          └─ getAvgRoutesPerSessionBySession.ts # route-stats 4.9 session
          └─ getReplayRatioBySession.ts         # route-stats 4.10 session
          └─ getTotalRoutesCompletedByDateRange.ts # route-stats 4.1 date-range
          └─ getTotalConnectedUsersByDateRange.ts # route-stats 4.2 date-range
          └─ getUsersStartedMoreThanOneRouteByDateRange.ts
          └─ getUsersCompletedMoreThanOneRouteByDateRange.ts

```

```

├── getUsersNeverStartedRouteByDateRange.ts
├── getUsersStartedButNotFinishedByDateRange.ts
├── getTotalReplaysByDateRange.ts
├── getAvgRoutesPerUserByDateRange.ts
├── getAvgRoutesPerSessionByDateRange.ts
├── getReplayRatioByDateRange.ts
├── Transactions/
│   └── getNextRouteForPlayerSession.service.ts
├── Update/
│   └── addChildrenadults.ts
└── tests/
    ├── unit/
    │   ├── middleware.test.ts
    │   └── campbell/
    │       ├── api.test.ts
    │       ├── api.jwt-guard.test.ts
    │       ├── api.utilities.test.ts
    │       ├── game.service.test.ts
    │       ├── route.service.test.ts
    │       ├── getNextRouteForPlayerSession.service.test.ts
    │       ├── CheckSession.service.test.ts
    │       ├── reshape.utilities.test.ts
    │       ├── prisma.test.ts
    │       └── index.test.ts
    ├── integration/
    │   └── routes.test.ts
    └── databaseOperations/
        ├── hasSession.test.ts
        ├── getOrCreatePlayerSession.test.ts
        ├── getActiveRouteForPlayerSession.test.ts
        ├── getChildrenAndAdultBySession.test.ts
        └── getChildNumAndAdultDateToForever.test.ts

```

2. Server Entry Point — `index.ts`

Path: [backend/src/index.ts](#)

This is the first file Node runs. Everything starts here.

```

import app from "./app";
import { APIUtilities } from "./api/Utilities/api.utilities";
import { prisma } from "./services/db/prisma";

```

Three imports: the configured Express app, the utility class that can generate session IDs, and the Prisma singleton for DB access.

```

const PORT = Number(process.env.PORT) || 4000;

```

Reads `PORT` from environment. If unset or not a valid number, defaults to `4000`. This is a number (not a string) because `app.listen` expects a number.

```

async function start() {

```

The startup logic runs inside an async function so it can `await` DB calls before the server begins accepting connections.

```

const sessionId = APIUtilities.generateSessionId();

```

Generates today's session ID in `YYMMDDNNN` format, Eastern timezone. Example: April 5, 2026 → `260405042` . The 3-digit suffix is a random number. See §7 for the exact implementation.

```
const existing = await prisma.session.findUnique({ where: { id: sessionId } });
if (!existing) {
  await prisma.session.create({ data: { id: sessionId, sessionId: new Date() } });
}
```

Checks if a `Session` row already exists for today's ID. If not, creates one. This is idempotent — if the server restarts mid-day, it won't create a duplicate session. The `sessionId` is set to `new Date()` (current UTC timestamp).

```
console.log("=====");
console.log(`  TODAY'S SESSION ID: ${sessionId}`);
console.log("=====\\n");
```

Prints the session ID prominently in the container logs. Staff check the logs to get the day's code.

```
app.locals.dailySessionId = sessionId;
```

Stores the session ID on the Express app instance for potential use by handlers. Currently informational — handlers derive the session from the JWT, not from `app.locals` .

```
app.listen(PORT, "0.0.0.0", () => console.log(`Listening on ${PORT}`));
```

Starts the HTTP server. `"0.0.0.0"` binds to all network interfaces — required inside Docker so the container is reachable from other containers and from the host port mapping. The callback fires after the port is open.

```
start().catch((err) => {
  console.error("STARTUP ERROR:", err);
  process.exit(1);
});
```

If anything in `start()` throws (DB unreachable, seed failure, etc.), the error is logged and the process exits with code 1. Docker will then restart the container if `restart: unless-stopped` is set.

3. Express App — `app.ts`

Path: [backend/src/app.ts](#)

Builds and exports the Express application object. Imported by `index.ts` and by integration tests.

```
import express from "express";
import cors from "cors";
import rateLimit from "express-rate-limit";
import { authenticateJWT } from "../middleware/middleware";
```

Core Express, the CORS middleware, the rate limiter, and the JWT authentication middleware.

```
import {
  Validate, startNewGame, completeRoute, createSession,
  addnumAdultsAndChildren, getnumchildrenandadultsbysessionId,
  getSessionsandPlayeridsbyChildSize, getSessionsandPlayeridsbyadultSize,
  getSessionsandPlayeridsbyfamilySize,
  getChildNumAndAdultDataByStartAndEndDate, getChildNumAndAdultDataToForever,
  getChildNumAndAdultDataToEndDate, polinatorNames,
  routeStatsTotalRoutesCompletedBySession, routeStatsTotalRoutesCompletedByDateRange,
```

```
// ... (all 20 route-stats handlers)
} from "./api/api";
```

All 34 handler functions are imported from `api.ts`. Each handler is a named export, one per route.

```
const app = express();
```

Creates the Express application instance. At this point it has no middleware or routes.

```
app.set("trust proxy", 1);
```

Tells Express to trust one level of proxy headers (`X-Forwarded-For`, `X-Forwarded-Proto`). Required when running behind Nginx — without this, `express-rate-limit` would see Nginx's IP for every request instead of the real client IP, making per-IP rate limiting useless.

```
const globalLimiter = rateLimit({ windowMs: 60_000, max: 500 });
const sessionLimiter = rateLimit({ windowMs: 60_000, max: 30 });
```

Two rate limiters. `globalLimiter` allows up to 500 requests per minute per IP across all `/api/*` routes. `sessionLimiter` is a tighter 30 req/min cap applied only to session creation to prevent automated session spam.

```
app.use(cors());
```

Applied globally. Allows requests from any origin. Adds the `Access-Control-Allow-Origin: *` response header. Without this, browser security would block the frontend (on port 80) from fetching the backend (on port 4000 or via the Nginx proxy at the same origin).

```
app.use(express.json());
```

Applied globally. Parses incoming `Content-Type: application/json` bodies and attaches the result to `req.body`. Without this, `req.body` would be `undefined` in all handlers.

```
app.use("/api", globalLimiter);
```

Applies the global limiter to every route that starts with `/api`. This middleware runs before any handler.

```
// Game Routes (player-facing)
app.post("/api", Validate);
app.post("/api/start-game", authenticateJWT, startNewGame);
app.post("/api/complete-route", authenticateJWT, completeRoute);
app.post("/api/create-session", sessionLimiter, createSession);
app.post("/api/add-children-adults", authenticateJWT, addnumAdultsAndChildren);
app.post("/api/get-pollinator-names", authenticateJWT, polinatorNames);
```

Six game routes. The three with `authenticateJWT` as the second argument require a valid Bearer token — the middleware runs first and calls `next()` only on success. `createSession` has `sessionLimiter` instead.

```
app.get("/api/health", (req, res) => {
  res.status(200).json({ status: "ok", timestamp: new Date().toISOString() });
});
```

Health check endpoint. Defined inline (no separate handler) because it's trivial. Returns the current UTC timestamp alongside the status string so uptime monitors can verify the response is fresh.

```
// Stats/Portal Routes (admin-facing, no auth)
app.post("/api/get-children-adults", getnumChildrenandadultsbysessionId);
```

```

app.post("/api/get-sessions-playerids-by-child-size", getSessionsandPlayeridsbyChildSize);
app.post("/api/get-sessions-playerids-by-family-size", getSessionsandPlayeridsbyfamilySize);
app.post("/api/get-sessions-playerids-by-adult-size", getSessionsandPlayeridsbyadultSize);
app.post("/api/get-child-adult-data-by-start-end-date", getChildNumAndAdultDataByStartAndEndDate);
app.post("/api/get-child-adult-data-to-forever", getChildNumAndAdultDataToForevers);
app.post("/api/get-child-adult-data-to-end-date", getChildNumAndAdultDataToEndDate);

```

Seven portal routes for demographic data. No `authenticateJWT` — these are internal staff tools, access is restricted at the firewall/Nginx level, not by token.

```

// Route Statistics APIs (20 endpoints)
app.post("/api/route-stats/total-routes-completed-by-session",
routeStatsTotalRoutesCompletedBySession);
app.post("/api/route-stats/total-routes-completed-by-date-range",
routeStatsTotalRoutesCompletedByDateRange);
app.post("/api/route-stats/total-connected-users-by-session",
routeStatsTotalConnectedUsersBySession);
app.post("/api/route-stats/total-connected-users-by-date-range",
routeStatsTotalConnectedUsersByDateRange);
app.post("/api/route-stats/users-started-more-than-one-route-by-session",
routeStatsUsersStartedMoreThanOneRouteBySession);
app.post("/api/route-stats/users-started-more-than-one-route-by-date-range",
routeStatsUsersStartedMoreThanOneRouteByDateRange);
app.post("/api/route-stats/users-completed-more-than-one-route-by-session",
routeStatsUsersCompletedMoreThanOneRouteBySession);
app.post("/api/route-stats/users-completed-more-than-one-route-by-date-range",
routeStatsUsersCompletedMoreThanOneRouteByDateRange);
app.post("/api/route-stats/users-never-started-route-by-session",
routeStatsUsersNeverStartedRouteBySession);
app.post("/api/route-stats/users-never-started-route-by-date-range",
routeStatsUsersNeverStartedRouteByDateRange);
app.post("/api/route-stats/users-started-but-not-finished-by-session",
routeStatsUsersStartedButNotFinishedBySession);
app.post("/api/route-stats/users-started-but-not-finished-by-date-range",
routeStatsUsersStartedButNotFinishedByDateRange);
app.post("/api/route-stats/total-replays-by-session", routeStatsTotalReplaysBySession);
app.post("/api/route-stats/total-replays-by-date-range", routeStatsTotalReplaysByDateRange);
app.post("/api/route-stats/avg-routes-per-user-by-session", routeStatsAvgRoutesPerUserBySession);
app.post("/api/route-stats/avg-routes-per-user-by-date-range",
routeStatsAvgRoutesPerUserByDateRange);
app.post("/api/route-stats/avg-routes-per-session-by-session",
routeStatsAvgRoutesPerSessionBySession);
app.post("/api/route-stats/avg-routes-per-session-by-date-range",
routeStatsAvgRoutesPerSessionByDateRange);
app.post("/api/route-stats/replay-ratio-by-session", routeStatsReplayRatioBySession);
app.post("/api/route-stats/replay-ratio-by-date-range", routeStatsReplayRatioByDateRange);

```

20 route statistics endpoints — 10 metrics × 2 scope modes (single session vs. date range). All unauthenticated, all returning the same envelope shape. See [ROUTE STATISTICS SPEC.md](#) for metric definitions.

```
export default app;
```

Exported as the default export so `index.ts` can import it and call `.listen()`, and integration tests can import it and pass it to `supertest`.

4. JWT Middleware — `middleware.ts`

Path: [backend/src/middleware/middleware.ts](#)

```
import { Request, Response, NextFunction } from "express";
import jwt from "jsonwebtoken";
```

Standard Express types and the `jsonwebtoken` library.

```
const JWT_SECRET = process.env.JWT_SECRET;
if (!JWT_SECRET) {
  throw new Error("JWT_SECRET environment variable is required");
}
```

This runs at **module load time** (when `app.ts` first imports `middleware.ts`). If `JWT_SECRET` is missing, the entire server startup fails immediately with a clear error message. This prevents the server from running in an insecure state where tokens cannot be verified.

```
export function authenticateJWT(req: Request, res: Response, next: NextFunction) {
```

Standard Express middleware signature. Applied to individual routes in `app.ts`.

```
  const auth = req.headers.authorization;
  if (!auth?.startsWith("Bearer ")) {
    res.status(401).json({ error: "Missing token" });
    return;
  }
```

Reads the `Authorization` header. If absent or not starting with `"Bearer "`, returns 401 immediately. The `?.` (optional chaining) handles `undefined` headers without throwing.

```
  try {
    const token = auth.slice("Bearer ".length);
```

Strips the `"Bearer "` prefix (7 characters) to get the raw JWT string.

```
    const payload = jwt.verify(token, JWT_SECRET, { algorithms: ["HS512"] }) as { sessionId: number;
    playerId: number };
```

Verifies the token's signature using `JWT_SECRET` and enforces the `HS512` algorithm. `jwt.verify` throws if:

- The signature doesn't match
- The token is expired (the JWT has a 12-hour `expiresIn`)
- The algorithm doesn't match Passing `algorithms: ["HS512"]` explicitly prevents algorithm substitution attacks.

```
    (req as any).auth = { sessionId: payload.sessionId, playerId: payload.playerId };
    next();
```

On success, attaches `sessionId` and `playerId` from the JWT payload to `req.auth`. Casts to `any` because Express's TypeScript types don't include a custom `auth` property. Calls `next()` to pass control to the actual handler. The handler always reads player identity from `req.auth`, never from `req.body`, preventing clients from claiming to be a different player.

```
  } catch {
    res.status(401).json({ error: "Invalid token" });
  }
```

Catches any error from `jwt.verify` (expired, tampered, wrong algorithm). Returns 401 without revealing why the token was rejected.

5. Type Definitions

route.dto.ts

Path: [backend/src/Back-end types/route.dto.ts](#)

```
export type RouteDTO = {
  id: number;
  name: string;
  sourcePath: unknown | null;
  nodes: Array<{ id: number; key: string; text: string; position: number }>;
  facts: Array<{ nodeId: string; text: string; pollinator: string }>;
};
```

The internal shape of a fully loaded route. This is what `LoadRouteDTOService.loadRouteDTO()` returns and what `GameService` works with internally. It is **never sent directly to the frontend** — it is always converted via `mapRouteDTOToLegacyRoute()` first.

- `id` — numeric DB primary key of the `Route` row
- `name` — the pollinator name (e.g. "Bee")
- `sourcePath` — the raw node key array stored in the DB (legacy reference data, not used at runtime)
- `nodes[].id` — the `RouteNode` primary key (numeric)
- `nodes[].key` — the coordinate string (e.g. "3.2") — this becomes `routeNodes[].id` in the frontend shape
- `nodes[].text` — the stop card text
- `nodes[].position` — ordering index within this route
- `facts[].nodeId` — the `FactNode.nodeId` string (e.g. "Bee.0")
- `facts[].text` — the fact text
- `facts[].pollinator` — the pollinator name for this fact

api.interfaces.ts

Path: [backend/src/api/interfaces/api.interfaces.ts](#)

TypeScript interfaces for request body shapes and response shapes, used as generic type parameters on Express `Request<>` and `Response<>` to get compile-time type checking in handler functions.

```
export interface ValidateRequestBody { sessionId?: any; }
export interface ValidateSuccessResponse { token: string; }
export interface ErrorResponse { error: string; message?: string; }

export interface StartNewGameResponse {
  sessionId: string;
  player: { id: any; routesCompleted: any[]; activeRoute: any | null; };
  availableRoutes: any[];
}

export interface CompleteRouteRequestBody { sessionId?: any; }
export interface SessionIdQuery { sessionId?: any; }

export type CompleteRouteResponse = {
  success: boolean;
  message: string;
  newActiveRoute: LegacyRoute | null;
  routesCompleted: number;
  completedPollinators: string[];
};
```

`LegacyRoute` is imported from `shared/types.ts` — the frontend-compatible shape. `availableRoutes` and `routesCompleted` on `StartNewGameResponse` are legacy fields retained for backward compatibility but not meaningfully populated.

6. API Handlers — `api.ts`

Path: `backend/src/api/api.ts`

All handler functions live in one file. The file also instantiates the `GameService` and declares the `JWT_SECRET` guard.

```
const gameService = new GameService(prisma);
```

A single `GameService` instance is created at module load time and shared across all requests. This is safe because `GameService` holds no mutable per-request state — it's a thin orchestrator over stateless service calls.

```
const JWT_SECRET = process.env.JWT_SECRET;
if (!JWT_SECRET) {
  throw new Error("JWT_SECRET environment variable is required");
}
```

A second guard (also in `middleware.ts`). `api.ts` uses `JWT_SECRET` directly for token signing in `Validate`. If it were missing and only `middleware.ts` had the guard, the server could start but `Validate` would crash at runtime.

Handler: `Validate` — `POST /api`

Issues a JWT to a player after validating their session code.

```
export async function Validate(
  req: Request<{}, any, ValidateRequestBody>,
  res: Response<ValidateSuccessResponse | ErrorResponse>
): Promise<void> {
```

Types the request body as `ValidateRequestBody` and constrains the response to either a success or error shape.

```
const sessionId = APIUtilities.toNumber(req.body.sessionId);
if (sessionId === null) {
  res.status(400).json({ error: "Session ID is required" });
  return;
}
```

Safely converts `req.body.sessionId` (which could be a string, number, or undefined) to a number. Returns 400 if it can't be parsed. The `return` after `res.json()` is critical — without it, Express would try to send a second response later and throw a "headers already sent" error.

```
const ok = await HasSessionService.hasSession(sessionId);
if (!ok) {
  res.status(401).json({ error: "Invalid session ID" });
  return;
}
```

Checks the `Session` table. The session must exist AND its `sessionDate` must be today (compared in server-local time). Returns 401 if not found or not today's session.

```
const playerId = await APIUtilities.generateUniquePlayerId(sessionId);
```

Generates a collision-free random integer between 1 and 2,000,000,000. Checks against the `PlayerSession` table for the given `sessionId` to ensure uniqueness. Retries up to 300 times on collision (astronomically unlikely with <1000 players per session).

```
const token = jwt.sign({ sessionId, playerId }, JWT_SECRET, { algorithm: "HS512", expiresIn: "12h" });
```

```
res.status(200).json({ token });
```

Signs a JWT containing `sessionId` and `playerId`. Algorithm `HS512` (HMAC-SHA-512). Token expires after 12 hours — long enough for an all-day event, short enough to limit exposure if stolen.

```
} catch (error) {
  console.error("Validate error:", error);
  const message = error instanceof Error ? error.message : "Unknown error";
  res.status(500).json({ error: "Wrong input", message });
}
```

Catches any unexpected DB or signing error. Logs the full error server-side, returns a safe message to the client.

Handler: `pollinatorNames` — `POST /api/get-pollinator-names`

Returns all pollinators with their spritesheet coordinates. JWT required.

```
const pollinatorNames = await GetpollinatorNames.getPollinatorNames();
const payload = {
  success: true,
  message: "data retrieved successfully",
  pollinator: pollinatorNames.map((p) => ({ name: p.pollinator, coord: p.nodeId })),
};
res.status(200).json(payload);
```

Queries all routes from the DB, finds each route's last node (highest position), and returns `{ name, coord }` pairs. The `coord` is the node `key` string (e.g. `"4.2"`) used by the frontend as a spritesheet offset. Used by both the home page (to count total pollinators) and the collection page (to show silhouettes).

Handler: `startNewGame` — `POST /api/start-game`

Gets or creates a player's session and returns their active route. JWT required.

```
const sessionId = APIUtilities.getSessionIdFromReq(req);
if (sessionId === null) {
  res.status(400).json({ error: "Session ID is required" });
  return;
}
const playerId = APIUtilities.getPlayerIdFromReq(req);
```

Extracts both values from `req.auth` (set by JWT middleware). `sessionId` is required; `playerId` may be `undefined` in edge cases (though the JWT should always contain it after `Validate`).

```
const started = await gameService.startOrJoin(sessionId, playerId);
const activeRoute = started.activeRoute
  ? ReshapeUtilities.mapRouteDTOToLegacyRoute(started.activeRoute)
  : null;
```

Delegates to `GameService.startOrJoin()` which creates/finds the `PlayerSession` and allocates a route. Converts the internal `RouteDT0` to the legacy `Route` shape the frontend understands. If `activeRoute` is `null` (all routes completed), it stays `null`.

```
const responseData: StartNewGameResponse = {
  sessionId: String(sessionId),
  player: { id: started.playerId, routesCompleted: [], activeRoute },
  availableRoutes: [],
```

```
};  
res.status(200).json(responseData);
```

`routesCompleted: []` and `availableRoutes: []` are legacy fields — the frontend ignores them. Only `player.id` and `player.activeRoute` are used.

Handler: `completeRoute` — `POST /api/complete-route`

Marks a route done and returns the next route. JWT required.

```
const result = await gameService.completeRouteAndAdvance(sessionId, playerId);  
const newActiveRoute = result.newActiveRoute  
  ? ReshapeUtilities.mapRouteDTOToLegacyRoute(result.newActiveRoute)  
  : null;  
  
const payload: CompleteRouteResponse = {  
  success: true,  
  message: "Route completed successfully",  
  newActiveRoute,  
  routesCompleted: result.routesCompleted,  
  completedPollinators: result.completedPollinators,  
};  
res.status(200).json(payload);
```

`completedPollinators` is an array of pollinator names for every route the player has finished — used by the frontend to render the collection screen. When `newActiveRoute` is `null`, the frontend knows the player has finished all routes.

```
} catch (error) {  
  const msg = error instanceof Error ? error.message : String(error);  
  if (msg === "No active route to complete") {  
    res.status(404).json({ error: msg });  
    return;  
  }  
  res.status(500).json({ error: "Failed to complete route", message: msg });  
}
```

The "No active route" error comes from `RouteService.completeActiveRouteForPlayerSession()` when no incomplete `RouteCycle` exists. This is a 404 (not found) rather than 500 (server error) because it's a known business logic case (e.g. double-submitting the complete button).

Handler: `createSession` — `POST /api/create-session`

Creates a new session record. No auth, rate-limited.

```
const existing = await prisma.session.findUnique({ where: { id: sessionId } });  
if (existing) {  
  res.status(200).json({ sessionId });  
  return;  
}  
await prisma.session.create({ data: { id: sessionId, sessionId: new Date() } });  
res.status(200).json({ sessionId });
```

Idempotent — returns 200 whether the session was just created or already existed. Used by IT staff to provision sessions outside the pre-seeded range. The 30/min rate limit prevents automated abuse.

Handler: `addnumAdultsAndChildren` — `POST /api/add-children-adults`

Records party size survey response. JWT required.

```
const { numAdults, numChildren } = req.body;
if (typeof numAdults !== "number" || typeof numChildren !== "number") {
  res.status(400).json({ error: "numAdults and numChildren must be numbers" });
  return;
}
await addChildrenAdultsService.addChildrenAdults(playerId, numChildren, numAdults, sessionId);
res.status(200).json({ success: true });
```

Validates both fields are numbers (not strings or null). Delegates to the update service which runs a single `UPDATE` on `PlayerSession`. The frontend only calls this once — after the first route completion. No enforcement server-side.

Handler: `getnumchildrenandadultsbysessionId` — `POST /api/get-children-adults`

Returns party size data for all players in a session. Portal use.

```
const session = await prisma.session.findUnique({ where: { id: sessionId }, select: { sessionId: true } });
if (!session) {
  res.status(404).json({ error: "Session not found" });
  return;
}
```

Fetches the session date for inclusion in the response. Returns 404 if the session doesn't exist (rather than empty data).

The response includes `sessionId`, and an array of `{ numChildren, numAdults, playerId }` — one row per player who submitted the survey. Players who didn't submit have `null` for both count fields.

Route Statistics Handlers (20 handlers)

All follow identical structure. Example — `routeStatsTotalRoutesCompletedBySession` :

```
export async function routeStatsTotalRoutesCompletedBySession(req, res): Promise<void> {
  try {
    const sessionId = APIUtilities.toNumber(req.body.sessionId);
    if (!sessionId) {
      res.status(400).json({ error: "sessionId or startDate is required" });
      return;
    }
    const value = await GetTotalRoutesCompleted.getroutescompleted(sessionId);
    res.status(200).json({
      success: true,
      scope: "session",
      scopeValue: sessionId,
      metric: "totalRoutesCompleted",
      value,
    });
  } catch (error) {
    res.status(500).json({ error: "Failed to compute statistics", message: String(error) });
  }
}
```

The date-range variant accepts `{ startDate, endDate }` instead of `{ sessionId }` and uses the corresponding `ByDateRange` DB service. All 20 handlers return the same envelope:

```
{ "success": true, "scope": "session"|"dateRange", "scopeValue": ..., "metric": "...", "value": ...
}
```

7. API Utilities

api.utilities.ts

Path: [backend/src/api/Utilities/api.utilities.ts](#)

toNumber(value: any): number | null

```
static toNumber(value: any): number | null {
  if (typeof value === "number" && isFinite(value)) return value;
  if (typeof value === "string") {
    if (value.trim() === "") return null;
    const n = Number(value);
    if (isFinite(n)) return n;
  }
  return null;
}
```

Used everywhere a request body field needs to be a number. Accepts actual numbers or numeric strings; rejects `NaN`, `Infinity`, empty strings, objects, arrays, `null`, and `undefined` by returning `null`.

getSessionIdFromReq(req: any): number | null

```
static getSessionIdFromReq(req: any): number | null {
  if (req?.auth?.sessionId === undefined) return null;
  return APIUtilities.toNumber(req.auth.sessionId);
}
```

Reads from `req.auth` (set by JWT middleware), not from `req.body`. The JWT is the authoritative source of the player's identity.

getPlayerIdFromReq(req: any): number | undefined

```
static getPlayerIdFromReq(req: any): number | undefined {
  if (req?.auth?.playerId === undefined) return undefined;
  const playerId = APIUtilities.toNumber(req.auth.playerId);
  if (playerId === null) return undefined;
  return playerId;
}
```

Same pattern. Returns `undefined` rather than `null` because `GameService.startOrJoin(sessionId, playerId?)` accepts `undefined` as "no player ID provided".

generateUniquePlayerId(sessionId: number): Promise<number>

```
static async generateUniquePlayerId(sessionId: number): Promise<number> {
  const MIN = 1;
  const MAX = 2_000_000_000;

  let attempts = 0;
  while (true) {
    const candidate = crypto.randomInt(MIN, MAX);

    const existing = await prisma.playerSession.findUnique({
      where: { sessionId_playerId: { sessionId, playerId: candidate } },
      select: { id: true },
    });

    if (!existing) return candidate;
  }
}
```

```

    if (++attempts >= 300) throw new Error("Failed to allocate unique playerId after many
attempts");
  }
}

```

`crypto.randomInt` (from Node's built-in `crypto` module) generates a cryptographically secure random integer in `[MIN, MAX)`. The DB check uses the composite unique index `sessionId_playerId`. With 2 billion possible values and at most ~1000 players per session, the collision probability on the first attempt is ~0.00005%. The 300-attempt guard exists as a safety net.

`generateSessionId(): number`

```

static generateSessionId(): number {
  const todayDate = new Date();
  const todayEasternTime = todayDate.toLocaleString("en-US", { timeZone: "America/New_York" });

  const todayYear = String(new Date(todayEasternTime).getFullYear()).slice(-2);
  const todayMonth = String(new Date(todayEasternTime).getMonth() + 1).padStart(2, "0");
  const todayDay = String(new Date(todayEasternTime).getDate()).padStart(2, "0");
  const randomNumber = String(Math.floor(Math.random() * 1000)).padStart(3, "0");
  return parseInt(`${todayYear}${todayMonth}${todayDay}${randomNumber}`, 10);
}

```

Converts today's date to Eastern timezone before extracting year/month/day — ensures the session ID is always based on the event's local date regardless of server timezone. `Math.random() * 1000` gives a 3-digit suffix (000–999). `padStart(2, "0")` and `padStart(3, "0")` ensure fixed widths. Result: a 9-digit integer like `260405042`.

`reshape.Utilites.ts`

Path: [backend/src/api/Utilities/reshape.Utilites.ts](#)

(Note: "Utilites" is a typo in the filename — intentionally preserved for backward compatibility.)

`mapRouteDT0ToLegacyRoute(route: any): LegacyRoute`

```

static mapRouteDT0ToLegacyRoute(route: any): LegacyRoute {
  const routeId = String(route.id);

  const routeNodes = (route.nodes ?? []).map((n: any) => {
    const id = String(n.key); // node's coordinate key, NOT its numeric DB id
    const text = String(n.text);
    return { id, text };
  });

  const factNodes = (route.facts ?? []).map((f: any) => {
    const id = String(f.nodeId); // FactNode.nodeId string (e.g. "Bee.0")
    const text = String(f.text);
    const pollinator = String(f.pollinator);
    return { id, text, pollinator };
  });

  return { routeId, routeNodes, factNodes };
}

```

Transforms `RouteDT0` into the legacy `Route` shape the frontend was originally built around:

RouteDT0 field	LegacyRoute field	Notes
<code>route.id</code> (number)	<code>routeId</code> (string)	Stringified — frontend uses string IDs
<code>node.key</code> (string, e.g. "3.2")	<code>routeNode.id</code>	The coordinate, NOT the numeric DB id

node.text	routeNode.text	Passed through
fact.nodeId (string, e.g. "Bee.0")	factNode.id	Passed through
fact.text	factNode.text	Passed through
fact.pollinator	factNode.pollinator	Passed through

The `?? []` null-coalescing on `nodes` and `facts` prevents crashes if a route has no nodes or facts loaded.

8. Service Guards

CheckSession.service.ts

Path: [backend/src/services/gameServiceUtils/CheckSession.service.ts](#)

```
export class checkSessionService {
  static async requireSession(sessionId: number): Promise<void> {
    if (!(await HasSessionService.hasSession(sessionId))) throw new Error("Invalid session ID");
  }
}
```

A single-method guard. Throws if `HasSessionService.hasSession()` returns false. Called at the start of `getOrCreatePlayerSession` before any DB writes — prevents creating `PlayerSession` rows for non-existent or expired sessions.

PlayeridTypeCheck.ts

Path: [backend/src/services/gameServiceUtils/PlayeridTypeCheck.ts](#)

```
export class PlayerIdTypeCheck {
  static requirePlayerId(playerId?: number): number {
    if (typeof playerId !== "number") throw new Error("Missing playerId");
    return playerId;
  }
}
```

A type guard that also coerces `playerId?` to `playerId` (non-optional). Throws if `playerId` is `undefined` or any non-number type. Returns the same value if valid, allowing it to be used as an inline assertion: `const finalPlayerId = PlayerIdTypeCheck.requirePlayerId(playerId)`.

9. Game Service — game.service.ts

Path: [backend/src/services/game.service.ts](#)

Orchestrates the two main game operations. Called directly by the `startNewGame` and `completeRoute` handlers.

```
export class GameService {
  constructor(private prisma: PrismaClient, private routes = new RouteService(prisma)) {}
}
```

Takes a `PrismaClient` and a `RouteService` instance. The default `routes = new RouteService(prisma)` means callers don't need to pass a `RouteService` manually — useful for production. Tests can inject a mock `RouteService` via the second argument.

startOrJoin(sessionId, playerId?)

```
async startOrJoin(
  sessionId: number,
```

```
playerId?: number
): Promise<{ playerId: number; activeRoute: RouteDTO | null }> {
```

```
const ps = await getOrCreatePlayerSessionService.getOrCreatePlayerSession(sessionId, playerId);
```

Step 1: Ensure a `PlayerSession` row exists. If `playerId` is provided and the `(sessionId, playerId)` pair doesn't exist yet, a new row is created. If it already exists (player rejoining), the existing row is returned. Idempotent.

```
let active = await
getActiveRouteForPlayerSessionService.getActiveRouteForPlayerSession(ps.playerSessionId);
if (active) {
  console.log(`[start-rejoin] playerId=${ps.playerId} ...`);
```

Step 2: Check for an already-active (incomplete) route. If the player closed the app mid-route and reopened it, `start-game` should resume where they left off, not assign a new route. The log line helps trace resumption events in production.

```
} else {
  const [snapshotCycles, allRoutes] = await Promise.all([
    this.prisma.routeCycle.findMany({
      where: { playerIdSessionId: ps.playerSessionId },
      select: { routeId: true, cycleNumber: true },
    }),
    this.prisma.route.findMany({ select: { id: true, name: true } }),
  ]);
```

Step 3a: If no active route, take a pre-allocation snapshot of the player's cycles and all available routes. This snapshot is used only for the debug log — it's fetched **before** the new route is allocated so the log accurately shows what was "seen" at selection time.

```
active = await getNextRouteForPlayerSessionService.getNextRouteForPlayerSession({
  sessionId,
  playerIdSessionId: ps.playerSessionId,
  requireNodes: true,
});
```

Step 3b: Allocate the next route via the transaction service. `requireNodes: true` ensures only routes that have actual stop content are assigned (prevents assigning empty routes if data is incomplete).

```
try {
  this.debugLogRouteAssignment({ event: "start", ... });
} catch (err) {
  console.error("[route-assign] logging failed:", err);
}
```

Step 4: Log the assignment. Wrapped in try/catch so a logging failure never crashes the game flow.

```
return { playerId: ps.playerId, activeRoute: active };
```

Returns the player's numeric ID and their current route (or null if none available).

completeRouteAndAdvance(sessionId, playerId?, completionMs?)

```
async completeRouteAndAdvance(
  sessionId: number,
  playerId?: number,
  completionMs?: number
```



```
) : Promise<{ playerId: number; routesCompleted: number; newActiveRoute: RouteDTO | null;
completedPollinators: string[] }> {
```

```
const ps = await getOrCreatePlayerSessionService.getOrCreatePlayerSession(sessionId, playerId);
```

Same first step as `startOrJoin` — idempotent player session lookup.

```
await this.routes.completeActiveRouteForPlayerSession({
  playerSessionId: ps.playerSessionId,
  completionMs,
});
```

Marks the current route complete: sets `wasCompleted = true`, `completedAt = now()`, and records `completionMs` if provided. Throws "No active route to complete" if no incomplete `RouteCycle` exists.

```
const [snapshotCycles, allRoutes] = await Promise.all([
  this.prisma.routeCycle.findMany({ where: { playerSessionId: ps.playerSessionId }, select: {
    routeId: true, cycleNumber: true } }),
  this.prisma.route.findMany({ select: { id: true, name: true } }),
]);
```

Pre-allocation snapshot for the debug log. Fetched **before** `getNextRoute` for the same reason as in `startOrJoin`.

```
const [routesCompleted, newActiveRoute, completedPollinators] = await Promise.all([
  this.routes.countCompletedRoutes(ps.playerSessionId),
  getNextRouteForPlayerSessionService.getNextRouteForPlayerSession({
    sessionId,
    playerSessionId: ps.playerSessionId,
    requireNodes: true,
  }),
  this.routes.getCompletedPollinators(ps.playerSessionId),
]);
```

Three operations run in parallel:

1. `countCompletedRoutes` — total `RouteCycle` rows where `wasCompleted = true`
2. `getNextRouteForPlayerSession` — allocates the next route (or returns null if all done)
3. `getCompletedPollinators` — names of all completed pollinators for the collection display

`Promise.all` runs them concurrently, saving two sequential round-trips.

debugLogRouteAssignment(params)

```
private debugLogRouteAssignment(params: { ... }): void {
  const latestCycle = snapshotCycles.reduce((max, r) => Math.max(max, r.cycleNumber), 1);
  const seenInCycle = new Set(
    snapshotCycles
      .filter(r => r.cycleNumber === latestCycle && r.routeId !== null)
      .map(r => r.routeId as number)
  );
  const allIds = allRoutes.map(r => r.id);
  const unseenIds = allIds.filter(id => !seenInCycle.has(id));

  const atCycleBoundary = unseenIds.length === 0;
  const isRepeat = newRouteId !== null && seenInCycle.has(newRouteId) && !atCycleBoundary;

  const tag = isRepeat ? "[REPEAT]" : "[route-assign]";
  console.log(`${tag} event=${event} playerId=${playerId} ... cycle=${latestCycle}${atCycleBoundary
```

```

? "-new" : ""} seen=[...] unseen=[...]`);

    if (isRepeat) {
        console.warn(` [REPEAT] routeId=${newRouteId} assigned while still in cycle ${latestCycle}...`);
    }
}

```

Calculates which routes were seen vs. unseen at the moment of assignment and emits a structured log line. Key fields:

- `cycle=N→new` — indicates a cycle boundary was crossed
- `seen=[1,3,5]` — route IDs seen in the current cycle (sorted)
- `unseen=[2,4,6]` — route IDs not yet seen
- `[REPEAT]` tag — emitted if a route was re-assigned while there were still unseen routes (indicates a bug)

10. Route Service — `route.service.ts`

Path: [backend/src/services/route.service.ts](#)

Low-level route lifecycle operations. Instantiated by `GameService` .

`completeActiveRouteForPlayerSession({ playerId, completionMs? })`

```

const active = await this.prisma.routeCycle.findFirst({
  where: {
    playerId: input.playerSessionId,
    completedAt: null, // not yet completed
    routeId: { not: null }, // has a route assigned (not a placeholder row)
  },
  orderBy: { id: "desc" }, // most recent assignment first
  select: { id: true },
});

if (!active) throw new Error("No active route to complete");

await this.prisma.routeCycle.update({
  where: { id: active.id },
  data: {
    completedAt: new Date(),
    completionMs: typeof input.completionMs === "number" ? input.completionMs : null,
    wasCompleted: true,
  },
});

```

Finds the most recent incomplete `RouteCycle` row for this player. The `orderBy: { id: "desc" }` ensures that if there are somehow multiple incomplete rows (shouldn't happen under normal operation), the newest one is completed first. The `completionMs` type-check prevents `null` being stored when `undefined` is passed.

`countCompletedRoutes(playerSessionId)`

```

return this.prisma.routeCycle.count({
  where: { playerId, wasCompleted: true },
});

```

Simple `COUNT(*)` filtered by `playerSessionId` and `wasCompleted` . Returns the total number of routes this player has completed across all cycles.

`getCompletedPollinators(playerSessionId)`

```
const cycles = await this.prisma.routeCycle.findMany({
  where: { playerId, wasCompleted: true },
  select: { route: { select: { name: true } } },
});
return cycles.flatMap(c => (c.route ? [c.route.name] : []));
```

Joins `RouteCycle` → `Route` to get route names. `flatMap` with the conditional filters out any `RouteCycle` rows where `route` is null (which can happen if a route was deleted after assignment — rare but handled).

getLatestCycleNumber(tx, playerId)

```
const latest = await tx.routeCycle.findFirst({
  where: { playerId },
  orderBy: { cycleNumber: "desc" },
  select: { cycleNumber: true },
});
return latest?.cycleNumber ?? 1;
```

Returns the highest `cycleNumber` seen for this player. Returns `1` if no `RouteCycle` rows exist yet — meaning the player starts in cycle 1. Takes a `tx` (transaction client) so it runs inside the allocation transaction atomically.

pickRandomRouteId(tx, opts)

```
const where = this.buildWhere(opts.requireNodes, opts.excludeSeenFor, opts.excludeRouteIds);
const count = await tx.route.count({ where });
if (count === 0) return null;
const skip = crypto.randomInt(0, count); // [0, count-1]
const picked = await tx.route.findFirst({
  where,
  orderBy: { id: "asc" },
  skip,
  select: { id: true },
});
return picked?.id ?? null;
```

Uniform random selection without loading all IDs into memory. `count` gives the total eligible routes; `randomInt(0, count)` gives an offset in `[0, count-1]`; `findFirst({ skip })` skips that many rows and returns the next one. `orderBy: { id: "asc" }` ensures a stable row ordering so the skip works correctly.

buildWhere(requireNodes, excludeSeenFor?, excludeRouteIds?)

```
const where: Prisma.RouteWhereInput = {};

if (requireNodes) {
  where.nodes = { some: {} }; // route must have at least one RouteNodeOnRoute row
}

if (excludeSeenFor) {
  where.cycles = {
    none: {
      playerId: excludeSeenFor.playerId,
      cycleNumber: excludeSeenFor.cycleNumber,
      routeId: { not: null },
    },
  };
}

if (excludeRouteIds && excludeRouteIds.length > 0) {
  where.routeId = { not: { in: excludeRouteIds } };
}
```

```

    where.id = { notIn: excludeRouteIds };
  }

```

The `cycles: { none: {...} }` clause is the deduplication guarantee: it translates to SQL `NOT EXISTS (SELECT 1 FROM RouteCycle WHERE playerSessionId=? AND cycleNumber=? AND routeId IS NOT NULL)` for each candidate route. Only routes with no `RouteCycle` record in the current cycle are eligible.

11. Prisma Client — `prisma.ts`

Path: [backend/src/services/db/prisma.ts](#)

```

import { PrismaClient, Prisma } from "../../../generated/prisma/client";

const url = process.env.DATABASE_URL!;
let adapter: any;

if (url.startsWith("postgres://") || url.startsWith("postgresql://")) {
  const { PrismaPg } = require("@prisma/adapter-pg");
  adapter = new PrismaPg(url);
} else {
  const { PrismaMariaDb } = require("@prisma/adapter-mariadb");
  adapter = new PrismaMariaDb(url);
}

export const prisma = new PrismaClient({ adapter });
export type { Prisma, PrismaClient };

```

Both adapters (`@prisma/adapter-pg` and `@prisma/adapter-mariadb`) are installed in `node_modules`. Only one is instantiated at runtime based on the `DATABASE_URL` prefix. The same generated client code (`generated/prisma/`) works with both adapters — the adapter is an abstraction layer. Exports `Prisma` (the namespace, used for types like `Prisma.TransactionClient`) and `PrismaClient` (the class, used for the type annotation in `GameService`).

12. DB Operation: `getOrCreatePlayerSession`

Path: [backend/src/services/db/DatabaseOperationsServices/Create/getOrCreatePlayerSession.service.ts](#)

```

static async getOrCreatePlayerSession(
  sessionId: number,
  playerId?: number
): Promise<{ playerSessionId: number; playerId: number }> {

```

```

  await checkSessionService.requireSession(sessionId);

```

Guard: throws `"Invalid session ID"` if the session doesn't exist in the DB or isn't today's date.

```

  const finalPlayerId = PlayerIdTypeCheck.requirePlayerId(playerId);

```

Guard: throws `"Missing playerId"` if `playerId` is not a number. After this line, `finalPlayerId` is guaranteed to be `number` (not `number | undefined`).

```

  await prisma.playerSession.createMany({
    data: [{ sessionId, playerId: finalPlayerId, startTime: new Date() }],
    skipDuplicates: true,
  });

```

Attempts to insert a new `PlayerSession` row. `skipDuplicates: true` silently ignores the insertion if `(sessionId, playerId)` already exists (the composite unique constraint is violated). This is the idempotency mechanism — calling `getOrCreatePlayerSession` twice for the same player has the same effect as calling it once.

```
const ps = await prisma.playerSession.findUniqueOrThrow({
  where: { sessionId_playerId: { sessionId, playerId: finalPlayerId } },
  select: { id: true, playerId: true },
});
return { playerSessionId: ps.id, playerId: ps.playerId };
```

Always fetches the row (whether just created or pre-existing) and returns its internal `id` (as `playerSessionId`) and `playerId`. Uses `findUniqueOrThrow` rather than `findUnique` — if the row somehow doesn't exist (impossible after the `createMany`), the thrown error is more descriptive than a null dereference.

13. DB Operation: `HasSessionService`

Path: <backend/src/services/db/DatabaseOperationsServices/Read/hassession.service.ts>

```
static async hasSession(sessionId: number): Promise<boolean> {
  const today = new Date();
  today.setHours(0, 0, 0, 0); // midnight of the current day (server time)

  const tomorrow = new Date(today);
  tomorrow.setDate(today.getDate() + 1); // midnight of the next day

  const session = await prisma.session.findUnique({
    where: { id: sessionId },
    select: { id: true, sessionDate: true },
  });

  if (!session) return false;

  const sessionDate = new Date(session.sessionDate);
  return sessionDate >= today && sessionDate < tomorrow;
}
```

Checks two things:

1. A `Session` row exists for the given `sessionId`.
2. `sessionDate` falls within today (midnight-to-midnight, server timezone).

The window `[today, tomorrow)` means a session created at any time today is valid. The comparison uses `>=` and `<` to be precise about the boundaries — `sessionDate < tomorrow` excludes tomorrow's midnight exactly. **Note:** This uses server-local time, not Eastern time (unlike `generateSessionId()`). If the server runs in UTC and the event is in Eastern time (UTC-5), a session created at 11pm Eastern (4am UTC the next day) would be checked against UTC midnight.

14. DB Operation: `getActiveRouteForPlayerSession`

Path: <backend/src/services/db/DatabaseOperationsServices/Read/getActiveRouteForPlayerSession.service.ts>

```
static async getActiveRouteForPlayerSession(playerSessionId: number): Promise<RouteDTO | null> {
  const active = await prisma.routeCycle.findFirst({
    where: {
      playerSessionId,
      completedAt: null, // route not yet completed
      routeId: { not: null }, // route is assigned (not a stub row)
    },
    orderBy: { id: "desc" },
    select: { routeId: true },
  });
}
```

```

});

if (!active?.routeId) return null;
return LoadRouteDT0Service.loadRouteDT0(active.routeId);
}

```

Finds the most recent incomplete `RouteCycle` for the player. If found, loads the full route data via `LoadRouteDT0Service`. Returns `null` if no active route exists (player needs a new one allocated).

15. DB Operation: `LoadRouteDT0Service`

Path: <backend/src/services/db/DatabaseOperationsServices/Read/LoadRouteDT0.service.ts>

```

static async loadRouteDT0(routeId: number): Promise<RouteDT0 | null> {
  const route = await prisma.route.findUnique({
    where: { id: routeId },
    select: {
      id: true,
      name: true,
      sourcePath: true,
      nodes: {
        select: {
          position: true,
          routeNode: { select: { id: true, key: true, text: true } },
        },
        orderBy: { position: "asc" }, // stops in display order
      },
      facts: {
        select: { nodeId: true, text: true, pollinator: true },
        orderBy: { nodeId: "asc" },
      },
    },
  });

  if (!route) return null;

  return {
    id: route.id,
    name: route.name,
    sourcePath: route.sourcePath ?? null,
    nodes: route.nodes.map((n) => ({
      id: n.routeNode.id,
      key: n.routeNode.key,
      text: n.routeNode.text,
      position: n.position,
    })),
    facts: route.facts.map((f) => ({
      nodeId: f.nodeId,
      text: f.text,
      pollinator: f.pollinator,
    })),
  };
}

```

A single Prisma query with nested selects loads a `Route` row, all of its `RouteNodeOnRoute` junction rows (with the linked `RouteNode` data), and all `FactNode` rows. The `orderBy: { position: "asc" }` on nodes is critical — it determines the order players see stops. Facts are ordered by `nodeId` (alphabetical, which effectively orders them within a pollinator). Returns `null` if the route ID doesn't exist.

16. DB Operation: GetPollinatorNames

Path: [backend/src/services/db/DatabaseOperationsServices/Read/GetpollinatorNames.ts](#)

```
static async getPollinatorNames() {
  const routes = await prisma.route.findMany({
    select: {
      name: true,
      nodes: {
        select: {
          position: true,
          routeNode: { select: { key: true } },
        },
        orderBy: { position: "desc" }, // highest position = last stop
        take: 1,                       // only the final node
      },
    },
  });

  return routes.map(r => ({
    pollinator: r.name,
    nodeId: r.nodes[0]?.routeNode.key ?? "0.0",
  }));
}
```

For each route, fetches only the last node (ordered by position descending, take 1). The last node's `key` is the spritesheet coordinate for the pollinator illustration. Returns `"0.0"` as a fallback if a route has no nodes (shouldn't happen in a properly seeded DB).

17. DB Operation: getNextRouteForPlayerSession (Transaction)

Path:

[backend/src/services/db/DatabaseOperationsServices/Transactions/getNextRouteForPlayerSession.service.ts](#)

This is the most complex service in the backend. It atomically selects a route and records it as "seen" in a single database transaction, with a retry loop for concurrency safety.

```
static async getNextRouteForPlayerSession(input: {
  sessionId: number;
  playerSessionId: number;
  requireNodes?: boolean;
}): Promise<RouteDTO | null> {
  const requireNodes = input.requireNodes !== false;
  const routeService = new RouteService(prisma);

  let attempt = 0;
  while (true) {
    if (attempt++ >= 300) break;
```

The `while(true)` loop with a 300-attempt counter is the retry mechanism. Normal execution completes on the first iteration. The counter prevents infinite loops in degenerate cases (e.g. all routes vanish from the DB between selection and load).

```
    let routeId: number | null;
    try {
      routeId = await prisma.$transaction(async (tx) => {
```

`prisma.$transaction` creates an interactive transaction. All Prisma calls inside the callback use `tx` (the transaction client) and are atomically committed when the callback returns, or rolled back if it throws.

```
const currentCycle = await routeService.getLatestCycleNumber(tx, input.playerSessionId);
```

Step A: Finds the highest `cycleNumber` in `RouteCycle` for this player. Returns 1 if no rows exist. This is the cycle number the player is currently working through.

```
let cycleNumber = currentCycle;
let picked = await routeService.pickRandomRouteId(tx, {
  requireNodes,
  excludeSeenFor: { playerId: input.playerSessionId, cycleNumber },
});
```

Step B: Attempts to pick a random route that hasn't been seen in the current cycle. The `excludeSeenFor` filter uses `cycles: { none: {...} }` to exclude any route that already has a `RouteCycle` row for this `(playerSessionId, cycleNumber)` pair. Returns `null` if no unseen routes remain.

```
if (!picked) {
  cycleNumber = currentCycle + 1;
  const recentCompleted = await tx.routeCycle.findMany({
    where: { playerId: input.playerSessionId, wasCompleted: true },
    orderBy: { completedAt: "desc" },
    take: 2,
    select: { routeId: true },
  });
  const excludeIds = recentCompleted
    .map((r) => r.routeId)
    .filter((id): id is number => id != null);

  picked = await routeService.pickRandomRouteId(tx, {
    requireNodes,
    excludeRouteIds: excludeIds.length > 0 ? excludeIds : undefined,
  });

  if (!picked) {
    picked = await routeService.pickRandomRouteId(tx, { requireNodes });
  }
}
```

Step C (cycle boundary): If all routes in the current cycle have been seen:

1. Increment `cycleNumber` to start a new cycle.
2. Fetch the 2 most recently completed routes (by `completedAt` DESC, limit 2).
3. Try to pick any route, excluding those 2 — this prevents the player from immediately seeing the same pollinator they just finished.
4. Fallback: if exclusions leave zero candidates (e.g. only 2 routes exist in the DB), relax all exclusions and pick any route.

```
if (!picked) return null;
```

If still no route (no routes in the DB at all), return `null` from the transaction. The outer function will return `null` to the caller, the game will show "no more routes."

```
await tx.routeCycle.create({
  data: {
    sessionId: input.sessionId,
    playerId: input.playerSessionId,
    routeId: picked,
    cycleNumber,
    startedAt: new Date(),
    wasCompleted: false,
  }
});
```



```

    },
    select: { id: true },
  });
  return picked;
});

```

Step D: Creates the `RouteCycle` row inside the transaction, recording the route as "seen" before the transaction commits. If another concurrent request picks the same `(playerSessionId, routeId, cycleNumber)` triplet, the `@unique` constraint causes a `P2002` error. `select: { id: true }` avoids loading the full row back.

```

} catch (e) {
  if (e != null && typeof e === "object" && "code" in e && e.code === "P2002") continue;
  throw e;
}

```

`P2002` is Prisma's error code for a unique constraint violation. When a concurrent request grabs the same route first, this `catch` block allows the outer `while` loop to retry. All other errors (DB connection issues, etc.) are re-thrown immediately.

```

if (!routeId) return null;

const dto = await LoadRouteDTOService.loadRouteDTO(routeId);
if (dto) return dto;
// Route vanished between selection and load – retry
}

throw new Error("Failed to allocate next route after retries");
}

```

After the transaction commits, loads the full route DTO. If `dto` is `null` (the route was deleted between the transaction committing and the `loadRouteDTO` call — extremely rare), the loop retries. After 300 failed attempts, throws.

18. DB Operation: `addChildrenAdultsService`

Path: <backend/src/services/db/DatabaseOperationsServices/Update/addChildrenadults.ts>

```

static async addChildrenAdults(
  playerId: number, numChildren: number, numAdults: number, sessionId: number
): Promise<void> {
  await prisma.playerSession.update({
    where: { sessionId_playerId: { sessionId, playerId } },
    data: { numChildren, numAdults },
  });
}

```

A single `UPDATE` on the `PlayerSession` table. Uses the composite unique key `sessionId_playerId` to target the exact row. Sets both `numChildren` and `numAdults` in one query. If the row doesn't exist (shouldn't happen since `start-game` always creates it first), Prisma throws a "Record to update not found" error.

19. Portal DB Operations (Read)

These are thin wrappers used by the 7 portal endpoints and 20 route-stats endpoints. They all follow the same pattern: validate input, run a Prisma query, return the result. Key ones are described here; see BACKEND_SERVICES.md for the full table.

`getChildrenandAdultsbySession`

Queries `PlayerSession` filtered by `sessionId`, selecting `numChildren`, `numAdults`, and `playerId`. Used by `POST /api/get-children-adults`.

Date-range portal queries

`getchilddadultdatabystartandend` , `getchilddnumandadulittoenddate` , `getchilddnumandaduldtetoforever` — filter `PlayerSession` by `createdAt` date range. The "forever" variant has no upper bound; the "to-end-date" variant has no lower bound.

Route Statistics queries (30 files)

Each file implements one specific metric:

- **Session scope** (10 files): Accept `sessionId` , filter `RouteCycle.sessionId = sessionId`
- **Date-range scope** (10 files): Accept `startDate` + optional `endDate` , join to `Session.sessionDate` for date filtering

All use either raw Prisma `count()` , `aggregate()` , or `groupBy()` . The `getTotalReplays` files query `COUNT(*) WHERE cycleNumber > 1` plus `COUNT(DISTINCT playerSessionId)` to return both total replay count and unique replay users. The `getReplayRatio` files divide these counts to produce `{ byPlayer: float, byRoute: float }` .

20. Database Schema — `schema.prisma`

Two schema files exist with identical models; only the `provider` and text column type differ:

	MySQL schema (<code>prisma/schema.prisma</code>)	PostgreSQL schema (<code>prisma/postgresql/schema.prisma</code>)
<code>datasource provider</code>	<code>"mysql"</code>	<code>"postgresql"</code>
<code>generator output</code>	<code>"../generated/prisma"</code>	<code>"../../generated/prisma"</code>
Large text columns	<code>@db.LongText</code>	<code>@db.Text</code>

Table: `Session`

```
model Session {
  id            Int      @id
  sessionDate   DateTime @default(now())
  totalRoutesCompleted Int?
  avgPlayerStartTime DateTime?
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  players      PlayerSession[]
  routeCycles  RouteCycle[]

  @@index([sessionDate])
}
```

Column	Type	Notes
<code>id</code>	<code>Int</code> (PK, not auto-increment)	The 9-digit public session code (e.g. 260405042). Staff share this with players. Manually assigned — never generated by the DB.
<code>sessionDate</code>	<code>DateTime</code>	Set to new <code>Date()</code> when the session is created by <code>index.ts</code> at startup or by the <code>POST /api/create-session</code> handler. The <code>HasSessionService</code> compares this to today's date.
<code>totalRoutesCompleted</code>	<code>Int?</code>	Legacy cached aggregate. Never reliably updated. Do not use this field — always compute live from <code>RouteCycle</code> .
<code>avgPlayerStartTime</code>	<code>DateTime?</code>	Legacy field. Never populated.
<code>createdAt</code>	<code>DateTime</code>	Auto-set by Prisma on insert.

updatedAt	DateTime	Auto-updated by Prisma on every write.
players	relation	One-to-many to PlayerSession.
routeCycles	relation	One-to-many to RouteCycle (denormalized for query performance).
@@index([sessionId])	—	Speeds up date-range queries in portal/route-stats endpoints.

Table: PlayerSession

```
model PlayerSession {
  id          Int          @id @default(autoincrement())
  playerId    Int
  sessionId   Int
  numChildren Int?
  numAdults   Int?
  startTime   DateTime?
  totalRoutesCompleted Json?
  routeCompletionTime Int?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  session Session @relation(fields: [sessionId], references: [id], onDelete: Cascade)
  cycles  RouteCycle[]

  @@unique([sessionId, playerId])
  @@index([sessionId])
  @@index([playerId])
}
```

Column	Type	Notes
id	Int (PK, auto-increment)	Internal ID used as playerId throughout the service layer.
playerId	Int	The player's device ID, embedded in their JWT. Generated by generateUniquePlayerId(). Range: 1–2,000,000,000.
sessionId	Int (FK → Session.id)	Which session this player is in. onDelete: Cascade — if the session row is deleted, all its PlayerSession rows are deleted too.
numChildren	Int?	Null until the party-size survey is submitted (POST /api/add-children-adults).
numAdults	Int?	Same.
startTime	DateTime?	Set to new Date() on row creation (in getOrCreatePlayerSession).
totalRoutesCompleted	Json?	Legacy field, never populated.
routeCompletionTime	Int?	Legacy field, never populated.
@@unique([sessionId, playerId])	—	Prevents two rows for the same player in the same session. Also used as the lookup key in findUnique({ where: { sessionId_playerId: {...} } }).
@@index([sessionId])	—	Speeds up queries filtering by session (portal endpoints).
@@index([playerId])	—	Speeds up queries filtering by player.

Table: RouteCycle

```

model RouteCycle {
  id          Int          @id @default(autoincrement())
  sessionId   Int
  playerSessionId Int
  routeId     Int?
  cycleNumber Int
  startedAt   DateTime?
  completedAt DateTime?
  completionMs Int?
  wasCompleted Boolean?
  meta        Json?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  session      Session      @relation(fields: [sessionId], references: [id], onDelete: Cascade)
  playerSession PlayerSession @relation(fields: [playerSessionId], references: [id], onDelete: Cascade)
  route        Route?       @relation(fields: [routeId], references: [id], onDelete: SetNull)

  @@index([sessionId])
  @@index([playerSessionId])
  @@index([routeId])
  @@unique([playerSessionId, routeId, cycleNumber])
}

```

Column	Type	Notes
id	Int (PK, auto-increment)	Internal row ID.
sessionId	Int (FK → Session.id)	Denormalized from PlayerSession.sessionId. Stored here to avoid a join in route-stats queries that filter by session. onDelete: Cascade.
playerSessionId	Int (FK → PlayerSession.id)	The player this assignment belongs to. onDelete: Cascade — if a PlayerSession is deleted, its cycles go too.
routeId	Int? (FK → Route.id)	Which route was assigned. Nullable because onDelete: SetNull — if the Route row is deleted, routeId becomes null rather than deleting the historical cycle record.
cycleNumber	Int	1 = first pass through all routes; 2 = first replay cycle; etc. Increments each time the player exhausts all routes in the current cycle.
startedAt	DateTime?	Set to new Date() when the route is assigned.
completedAt	DateTime?	Set when completeActiveRouteForPlayerSession runs. Null = route in progress or abandoned.
completionMs	Int?	Milliseconds from assignment to completion. Set alongside completedAt.
wasCompleted	Boolean?	true after completion, false at assignment time. Used for all "completed routes" queries.
meta	Json?	Reserved field — never read or written by any current code.
@@unique([playerSessionId, routeId, cycleNumber])	—	The concurrency safety constraint. Prevents two transactions from assigning the same route to the same player in the same cycle. P2002 violations trigger the retry loop in getNextRouteForPlayerSession.

@@index([sessionId])	—	Critical for route-stats queries filtering by session.
@@index([playerSessionId])	—	Critical for per-player queries (completeActive, count, etc.).
@@index([routeId])	—	Used by Prisma for the route relation join.

Table: Route

```
model Route {
  id          Int    @id @default(autoincrement())
  name        String @unique
  sourcePath  Json?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  nodes       RouteNodeOnRoute[]
  facts       FactNode[]
  cycles      RouteCycle[]
}
```

Column	Type	Notes
id	Int (PK, auto-increment)	Used as FK in RouteCycle.routeId and RouteNodeOnRoute.routeId.
name	String (unique)	The pollinator name (e.g. "Bee", "Hummingbird"). Unique — one route per pollinator. Also used as the pollinator field in FactNode.
sourcePath	Json?	The original array of node keys for this route (e.g. ["0","1.1","2.3","3.6","4.7","5.4","6.0"]). Set by the seed script. Used for reference/auditing; not queried at runtime.
nodes	relation	RouteNodeOnRoute[] — the ordered stops on this route.
facts	relation	FactNode[] — the facts associated with this route.
cycles	relation	RouteCycle[] — all game assignments of this route.

11 routes are seeded: Human, Bat, Hummingbird, Sunbird, Moth, Butterfly, Fly, Bee, Wasp, Beetle, Ant.

Table: RouteNode

```
model RouteNode {
  id          Int    @id @default(autoincrement())
  key         String @unique
  text        String @db.LongText // @db.Text in PostgreSQL
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  routes      RouteNodeOnRoute[]
}
```

Column	Type	Notes
id	Int (PK, auto-increment)	Internal ID. Not what the frontend uses as routeNode.id.
key	String (unique)	The spritesheet coordinate key, e.g. "0", "1.0", "3.2". Format: "row.column" (single-digit row for the root node "0"). This is what the frontend calls routeNode.id.

text	String (LongText)	The interpretive text shown at this stop. Can be long — uses LongText in MySQL, Text in PostgreSQL.
routes	relation	RouteNodeOnRoute[] — which routes include this node. A single RouteNode can be shared across multiple routes.

Why key and not id ? The frontend was built to use the coordinate string as the display identifier (for spritesheet offset calculation). The numeric `id` is only used internally for DB joins.

Table: RouteNodeOnRoute

```
model RouteNodeOnRoute {
  routeId      Int
  routeNodeId  Int
  position      Int

  route      Route      @relation(fields: [routeId], references: [id], onDelete: Cascade)
  routeNode  RouteNode @relation(fields: [routeNodeId], references: [id], onDelete: Cascade)

  @@id([routeId, routeNodeId])
  @@unique([routeId, position])
  @@index([routeNodeId])
}
```

Junction table connecting `Route` to `RouteNode` with an ordering position.

Column	Type	Notes
routeId	Int (FK → Route.id)	Part of composite PK. onDelete: Cascade — if the route is deleted, the junction rows go too.
routeNodeId	Int (FK → RouteNode.id)	Part of composite PK. onDelete: Cascade.
position	Int	Zero-indexed ordering of this node within the route. LoadRouteDTOService orders by position ASC to build the stops in the correct walking order.
@@id([routeId, routeNodeId])	—	Composite primary key — a node can only appear once per route.
@@unique([routeId, position])	—	No two nodes in the same route can have the same position. Enforces ordering integrity.
@@index([routeNodeId])	—	Speeds up lookups by node (used by Prisma relation resolution).

Table: FactNode

```
model FactNode {
  id          Int      @id @default(autoincrement())
  nodeId      String  @unique
  pollinator  String
  text        String  @db.LongText // @db.Text in PostgreSQL
  routeId     Int?
  route       Route?  @relation(fields: [routeId], references: [id], onDelete: SetNull)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([pollinator])
}
```

```
    @@index([routeId])
  }
}
```

Column	Type	Notes
id	Int (PK, auto-increment)	Internal ID.
nodeId	String (unique)	String identifier in format "Pollinator.index", e.g. "Bee.0", "Bee.6". Used as the factNode.id in the frontend shape.
pollinator	String	The pollinator name this fact belongs to (e.g. "Bee"). Matches Route.name. Indexed for queries filtering by pollinator.
text	String (LongText)	The fact text shown to the player after each route stop.
routeId	Int? (FK → Route.id)	Optional association to a route. onDelete: SetNull — if the route is deleted, the fact remains but routeId becomes null.
@@index([pollinator])	—	For queries filtering facts by pollinator name.
@@index([routeId])	—	For the route relation join.

Note: `nodeId` uses `pollinator.index` format, not `row.column` like `RouteNode.key`. These are two completely different ID namespaces. The `nodeId` `"Bee.0"` is the first fact about bees — it has no spritesheet coordinate meaning.

21. Seed Data — `seed.js` / `seed.pg.js`

Paths: [backend/prisma/seed.js](#) (MySQL), [backend/prisma/seed.pg.js](#) (PostgreSQL — identical logic, different string escaping for apostrophes).

Run by the `migrate` Docker service after `prisma migrate deploy`. Uses `upsert` throughout so re-running is safe.

Sessions (SESSIONS array)

```
const SESSIONS = [
  202401010, 202401020, ..., 202412310, // 365 entries: every day of 2024
];
```

One session ID per day of 2024. Format: `YYYYMMDD0` — note the trailing `0` (the NNN suffix is always `0` for seed sessions). The seed loop:

```
for (const sid of SESSIONS) {
  await prisma.session.upsert({
    where: { id: sid },
    update: {}, // no-op if already exists
    create: { id: sid }, // sessionId defaults to now()
  });
}
```

365 upserts. `update: {}` means "if the row exists, change nothing." Only creates if absent.

Survey Responses (development fixtures)

```
const surveyResponses = [
  { sessionId: SESSIONS[0], playerId: 1, numAdults: 5, numChildren: 4, createdAt: new Date("2026-03-01T12:00:00.000Z") },
  // ... 7 more rows across 4 sessions
];
```

8 fixture rows for development/testing — two players per session across the first 4 seeded sessions. Allows portal queries to return data without needing real event traffic. Uses `upsert` with explicit `createdAt` / `updatedAt` so the dates are deterministic.

Route Nodes (ROUTE_NODES object)

```
const ROUTE_NODES = {
  "0": { text: "Start" },
  "1": { "0": { text: "Two legs" }, "1": { text: "Six legs" } },
  "2": { "0": { text: "Mammal" }, "1": { text: "Not A Mammal" }, "2": { text: "Workers Have No Wings" }, "3": { text: "Wings" } },
  "3": { "0": { text: "No Wings" }, "1": { text: "Wings" }, "2": { text: "Can Hover In Flight" }, "3": { text: "Must Perch To Sip Nectar" }, "4": { text: "Ant" }, "5": { text: "Scales On Wings" }, "6": { text: "Membranous Wings" }, "7": { text: "Elytra (Hardened Covering For Hind Wings)" } },
  "4": { "0": { text: "Human" }, "1": { text: "Rubbery Wings" }, "2": { text: "Hummingbird" }, "3": { text: "Sunbird" }, "4": { text: "Feathery Antennae" }, "5": { text: "Filamentous Antennae" }, "6": { text: "Two Wings" }, "7": { text: "Four Wings" }, "8": { text: "Beetle" } },
  "5": { "0": { text: "Bat" }, "1": { text: "Moth" }, "2": { text: "Butterfly" }, "3": { text: "Fly" }, "4": { text: "Hairy Abdomen" }, "5": { text: "Smooth Abdomen" } },
  "6": { "0": { text: "Bee" }, "1": { text: "Wasp" } },
};
```

A nested object representing the decision tree. Keys are row numbers (outer) and column numbers (inner). The text values are the stop-card labels shown during gameplay. `flattenRouteNodes()` converts this nested structure into a flat array of `{ key, text }` objects:

```
function flattenRouteNodes(routeNodes) {
  const out = [];
  for (const [topKey, value] of Object.entries(routeNodes)) {
    if (value.text) {
      out.push({ key: topKey, text: value.text }); // root node "0"
    } else {
      for (const [subKey, subVal] of Object.entries(value)) {
        out.push({ key: `${topKey}.${subKey}`, text: subVal.text }); // e.g. "1.0", "3.6"
      }
    }
  }
  return out;
}
```

Each flattened node is upserted into `RouteNode` by `key` :

```
await prisma.routeNode.upsert({ where: { key: n.key }, update: { text: n.text }, create: { key: n.key, text: n.text } });
```

Routes (ROUTES object)

```
const ROUTES = {
  Human: ["0", "1.0", "2.0", "3.0", "4.0"],
  Bat: ["0", "1.0", "2.0", "3.1", "4.1", "5.0"],
  Hummingbird: ["0", "1.0", "2.1", "3.2", "4.2"],
  Sunbird: ["0", "1.0", "2.1", "3.3", "4.3"],
  Moth: ["0", "1.1", "2.1", "3.5", "4.4", "5.1"],
  Butterfly: ["0", "1.1", "2.3", "3.5", "4.5", "5.2"],
```



```

Fly:      ["0", "1.1", "2.3", "3.6", "4.6", "5.3"],
Bee:      ["0", "1.1", "2.3", "3.6", "4.7", "5.4", "6.0"],
Wasp:     ["0", "1.1", "2.3", "3.6", "4.7", "5.5", "6.1"],
Beetle:   ["0", "1.1", "2.3", "3.7", "4.8"],
Ant:      ["0", "1.1", "2.2", "3.4"],
};

```

Each key is a pollinator name mapping to an ordered array of `RouteNode.key` values. This defines the decision tree path a player walks to identify that pollinator. `"0"` is always the shared starting node.

For each route:

1. Upsert the `Route` row by `name`, storing the key array as `sourcePath`.
2. Delete all existing `RouteNodeOnRoute` rows for this route (to handle re-seeding after route changes).
3. Re-insert junction rows with the correct positions:

```

for (let i = 0; i < nodeKeys.length; i++) {
  const node = await prisma.routeNode.findUnique({ where: { key: nodeKeys[i] } });
  if (!node) throw new Error(`Route "${routeName}" references missing RouteNode key "${key}"`);
  await prisma.routeNodeOnRoute.create({
    data: { routeId: route.id, routeNodeId: node.id, position: i },
  });
}

```

The `throw` on missing node keys is a data integrity guard — if a route references a key that doesn't exist in `ROUTE_NODES`, the seed fails loudly rather than silently skipping it.

Facts (FACTS object)

```

const FACTS = {
  Human: [
    "Plant characteristics can include flower color, improved flavor...",
    "Hand pollination is usually an option only on a small scale.",
    "Plant breeders pollinate some crops by hand...",
    "In Maoxian County in southwestern China, apples are hand pollinated.",
  ],
  Bat: [
    "Do you like eating bananas and mangos? Bats help pollinate these fruits and more.",
    // ... 4 more
  ],
  // ... all 11 pollinators
};

```

Each pollinator maps to an array of fact strings. Facts are displayed sequentially during route play — one per stop (though in practice the same fact is shown at each stop for the given pollinator route). Each fact is seeded as a `FactNode`:

```

for (const [pollinator, facts] of Object.entries(FACTS)) {
  const route = await prisma.route.findUnique({ where: { name: pollinator } });
  for (let i = 0; i < facts.length; i++) {
    await prisma.factNode.upsert({
      where: { nodeId: `${pollinator}.${i}` },
      update: { text: facts[i], pollinator, routeId: route?.id ?? null },
      create: { nodeId: `${pollinator}.${i}`, text: facts[i], pollinator, routeId: route?.id ?? null },
    });
  }
}

```

`nodeId` format: "Bee.0" through "Bee.6" for 7 bee facts. The `routeId` links each fact to its route — `null` if the route doesn't exist (shouldn't happen after routes are seeded first).

22. Docker Configurations

`dockerfile.dev` (development image — shared by MySQL and PG dev stacks)

```
FROM node:24.14.1-alpine
RUN apk add --no-cache openssl
```

Uses Alpine Linux for a small image. `openssl` is required by Prisma's engine.

```
WORKDIR /app
COPY package*.json ./
COPY backend/package*.json backend/
COPY frontend/package*.json frontend/
COPY portal/package*.json portal/
RUN mkdir -p shared && echo '{"name":"shared","version":"0.0.1","private":true}' >
shared/package.json
RUN npm install
```

Installs all workspace dependencies. The stub `shared/package.json` prevents npm from failing on the shared workspace package, which has no `package.json` of its own.

```
WORKDIR /app/backend
COPY backend .
ENV CHOKIDAR_USEPOLLING=true
EXPOSE 4000
CMD ["npm", "run", "dev"]
```

Sets the working directory to `backend/`, copies source, enables Chokidar polling (required for file watching on Docker volume mounts on some host OSes), and runs `npm run dev` (`tsx watch src/index.ts` — TypeScript watch mode with auto-restart on file changes).

Dockerfile (MySQL production backend)

```
FROM node:24.14.1-slim AS deps
WORKDIR /app
COPY package.json package-lock.json ./
COPY backend/package.json ./backend/package.json
RUN npm ci
```

Stage 1: deps. Installs dependencies using `npm ci` (clean install from lockfile, reproducible). Only copies `package.json` files first to leverage Docker layer caching — this layer only rebuilds when dependencies change.

```
FROM node:24.14.1-slim AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY package.json package-lock.json ./
COPY tsconfig.base.json ./tsconfig.base.json
COPY shared/ ./shared/
COPY backend/ ./backend/

WORKDIR /app/backend
ENV NODE_ENV=production
RUN DATABASE_URL="mysql://user:pass@localhost:3306/db" \
  SHADOW_DATABASE_URL="mysql://user:pass@localhost:3306/shadowdb" \
```

```
npx prisma generate
RUN npm run build
```

Stage 2: build. Copies source and runs `prisma generate` with dummy database credentials (Prisma only needs the URL format at generate time, not a real connection). Then compiles TypeScript to JavaScript (`tsc` → `dist/`).

```
FROM node:24.14.1-slim AS prune
WORKDIR /app
COPY --from=build /app/node_modules ./node_modules
...
RUN npm prune --omit=dev
```

Stage 3: prune. Removes dev dependencies from `node_modules` . This keeps the final image small by excluding test frameworks, TypeScript, etc.

```
FROM gcr.io/distroless/nodejs24-debian12:nonroot AS runtime
WORKDIR /app
ENV NODE_ENV=production

COPY --from=prune --chown=65532:65532 /app/node_modules ./node_modules
COPY --chown=65532:65532 package.json ./
COPY --chown=65532:65532 backend/package.json ./backend/package.json
COPY --from=build --chown=65532:65532 /app/backend/dist ./backend/dist
COPY --from=build --chown=65532:65532 /app/backend/generated ./backend/generated

EXPOSE 4000
CMD ["backend/dist/backend/src/index.js"]
```

Stage 4: runtime. Uses Google's distroless image — no shell, no package manager, minimal attack surface. Runs as the non-root user `65532` (`--chown=65532:65532`). Only copies the compiled JavaScript (`dist/`), the generated Prisma client (`generated/`), and production `node_modules` . Does not include TypeScript source.

Dockerfile.pg (PostgreSQL production backend)

Identical to `Dockerfile` with one difference:

```
RUN npx prisma generate --config prisma.config.pg.ts
```

Uses the PostgreSQL Prisma config instead of the default MySQL one, generating the client against the PostgreSQL schema.

docker-compose.dev.yml (MySQL development stack)

```
services:
  backend:
    build:
      context: .
      dockerfile: backend/dockerfile.dev
    working_dir: /app/backend
    command: sh -c "npx prisma generate && npm run dev"
    ports:
      - "127.0.0.1:4000:4000"
    environment:
      DATABASE_URL: "mysql://pollinator:pollinatorpw@mysql:3306/pollinator"
      SHADOW_DATABASE_URL: "mysql://pollinator:pollinatorpw@mysql:3306/pollinator_shadow"
      JWT_SECRET: ${JWT_SECRET}
    volumes:
      - ./app
      - /app/node_modules
      - /app/backend/node_modules
```

```

depends_on:
  mysql:
    condition: service_healthy
  migrate:
    condition: service_completed_successfully

```

- `command: sh -c "npx prisma generate && npm run dev"` — generates the Prisma client on container start (so code changes to schema are picked up), then starts the dev server.
- `ports: "127.0.0.1:4000:4000"` — binds only to localhost; backend is not directly exposed on the network (only via Nginx in prod; dev uses the port directly from the host).
- Volume mounts: the entire repo is mounted at `/app` (enabling hot reload), with anonymous volume overrides for `node_modules` directories to prevent host `node_modules` from overwriting container `node_modules`.
- `depends_on migrate: service_completed_successfully` — waits for the migrate service to finish before starting the backend.

```

mysql:
  image: mysql:8.4.8
  environment:
    MYSQL_ROOT_PASSWORD: rootpw
    MYSQL_DATABASE: pollinator
    MYSQL_USER: pollinator
    MYSQL_PASSWORD: pollinatorpw
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-uroot", "-prootpw"]
    interval: 5s
    timeout: 3s
    retries: 30

```

MySQL with a health check. The backend's `depends_on mysql: service_healthy` waits for this health check to pass before starting. 30 retries × 5s = up to 2.5 minutes for MySQL to initialize.

```

migrate:
  build:
    context: .
    dockerfile: backend/dockerfile.dev
  restart: "no"
  command: sh -c "npx prisma migrate deploy && npx prisma generate && npx tsx prisma/seed.js"
  depends_on:
    mysql:
      condition: service_healthy

```

The migrate service runs once, exits 0 on success. Runs three steps sequentially: apply pending migrations, generate the Prisma client, run the seed script. `restart: "no"` prevents Docker from restarting it after it exits.

`docker-compose.dev.pg.yml` (PostgreSQL development stack)

Same structure as the MySQL dev stack with:

- `postgres` service instead of `mysql` (image: `postgres:18`, healthcheck: `pg_isready`)
- `DATABASE_URL: "postgresql://pollinator:pollinatorpw@postgres:5432/pollinator"` (no `SHADOW_DATABASE_URL` needed for PG)
- `command: sh -c "npx prisma generate --config prisma.config.pg.ts && npm run dev"`
- Migrate command: `npx prisma migrate deploy --config prisma.config.pg.ts && npx prisma generate --config prisma.config.pg.ts && npx tsx prisma/seed.pg.js`
- Volume mounts are more selective — only source directories are mounted (not the entire repo), which is faster on macOS

```

postgres:
  image: postgres:18

```

```
command: postgres -c max_connections=500
```

`max_connections=500` overrides PostgreSQL's default of 100. Required because Prisma opens a connection pool, and multiple concurrent requests plus the dev tools can exhaust connections quickly during development.

docker-compose.prod.pg.yml (PostgreSQL production stack)

```
postgres:
  image: postgres:18
  command: postgres -c max_connections=500
  env_file:
    - .env.pg.example
  volumes:
    - pollinator-postgres-data:/var/lib/postgresql
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER -d $$POSTGRES_DB"]
```

Uses a named volume (`pollinator-postgres-data`) for persistent data across container restarts. `env_file` loads credentials from `.env.pg.example` (or a `.env.pg` copy with real secrets).

```
migrate:
  build:
    context: .
    dockerfile: backend/Dockerfile.pg
    target: build # ← uses only the build stage, not the runtime distroless stage
  restart: "no"
  env_file: .env.pg.example
  command: sh -c "npx prisma migrate deploy --config prisma.config.pg.ts && npx tsx
prisma/seed.pg.js"
  depends_on:
    postgres:
      condition: service_healthy
```

Reuses the `build` stage of `Dockerfile.pg` (which has Node and the full source) rather than the distroless runtime stage (which has no shell). This avoids maintaining a separate Dockerfile for migrations.

```
backend:
  build:
    context: .
    dockerfile: backend/Dockerfile.pg # builds the full 4-stage distroless image
  restart: unless-stopped
  env_file: .env.pg.example
  depends_on:
    postgres:
      condition: service_healthy
    migrate:
      condition: service_completed_successfully
```

Backend only starts after both `postgres` is healthy AND `migrate` exits 0. No ports exposed — only Nginx communicates with the backend.

```
frontend:
  environment:
    NEXT_PUBLIC_API_URL: "" # empty = relative URLs (/api/...) proxied by Nginx
  healthcheck:
    test: ["CMD", "/nodejs/bin/node", "-e",
"require('http').get('http://localhost:3000/',...).on('error',...)"]
```

Distroless image has no `curl` or `wget`, so the health check uses Node's built-in `http` module inline.

```
nginx:
  image: nginx:stable-alpine
  ports:
    - "80:80"      # player-facing frontend + API
    - "3001:3001"  # staff portal + API
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
  depends_on:
    frontend:
      condition: service_healthy
    portal:
      condition: service_healthy
    backend:
      condition: service_started
```

Nginx is the sole public-facing component. It starts after frontend and portal are healthy (accepting connections) and backend has started.

23. Nginx Reverse Proxy

Path: [nginx/nginx.conf](#)

Two `server` blocks — one for the player app (port 80) and one for the portal (port 3001).

```
server {
  listen 80;
  server_name _;

  resolver 127.0.0.11 valid=10s;
```

`resolver 127.0.0.11` is Docker's internal DNS server. `valid=10s` means Nginx re-resolves upstream names every 10 seconds. This is critical — without it, Nginx caches the IP at startup and stops routing correctly if a container restarts and gets a new IP.

```
location = /api {
  set $backend_upstream http://backend:4000;
  proxy_pass $backend_upstream$request_uri;
  proxy_http_version 1.1;
  proxy_set_header Host $host;
  proxy_set_header X-Real-IP $remote_addr;
  proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  proxy_set_header X-Forwarded-Proto $scheme;
}

location /api/ {
  set $backend_upstream http://backend:4000;
  proxy_pass $backend_upstream$request_uri;
  ...
}
```

Two separate `location` blocks handle `POST /api` (exact match, no trailing slash — this is where `Validate` lives) and `GET/POST /api/*` (prefix match). Using a variable (`$backend_upstream`) forces Nginx to use the `resolver` for DNS lookups on every request rather than caching at config load time. `$request_uri` preserves the full path and query string.

```
location / {
  set $frontend_upstream http://frontend:3000;
  proxy_pass $frontend_upstream;
```

```

    ...
  }
}

```

All non-API requests go to the Next.js frontend container.

```

server {
  listen 3001;
  server_name _;
  resolver 127.0.0.11 valid=10s;

  location = /api { ... } # proxies to backend:4000
  location /api/ { ... } # proxies to backend:4000

  location / {
    set $portal_upstream http://portal:3000;
    proxy_pass $portal_upstream;
    ...
  }
}

```

The second server block serves the portal on port 3001. API routes proxy to the same backend:4000. All other requests go to the portal Next.js container.

24. Request Lifecycle — End to End

Game Request (e.g. `POST /api/complete-route`)

```

Player's Phone
| POST /api/complete-route
| Authorization: Bearer <JWT>
| Content-Type: application/json
|
▼
Nginx :80 (nginx.conf, server block 1)
| location /api/
| proxy_pass http://backend:4000 (Docker DNS)
| forwards X-Real-IP, X-Forwarded-For, Host
|
▼
Express app (app.ts)
| app.use("/api", globalLimiter) - checks 500 req/min per IP
| app.post("/api/complete-route", authenticateJWT, completeRoute)
|
▼
authenticateJWT (middleware.ts)
| strips "Bearer ", calls jwt.verify(token, JWT_SECRET, { algorithms: ["HS512"] })
| on success: req.auth = { sessionId, playerId }; next()
| on failure: 401
|
▼
completeRoute handler (api.ts)
| sessionId = getSessionIdFromReq(req) ← from req.auth (JWT)
| playerId = getPlayerIdFromReq(req) ← from req.auth (JWT)
| result = await gameService.completeRouteAndAdvance(sessionId, playerId)
|
▼
GameService.completeRouteAndAdvance (game.service.ts)
| 1. getOrCreatePlayerSession(sessionId, playerId)
|   → checkSessionService.requireSession(sessionId)
|     → HasSessionService.hasSession(sessionId) [DB: SELECT Session]
|   → playerIdTypeCheck.requirePlayerId(playerId)
|   → prisma.playerSession.createMany(..., skipDuplicates: true) [DB: INSERT]
|   → prisma.playerSession.findUniqueOrThrow(...) [DB: SELECT]
|   ← { playerIdSessionId, playerId }

```

```

2. routes.completeActiveRouteForPlayerSession(playerSessionId)
   → prisma.routeCycle.findFirst(where: { completedAt: null }) [DB: SELECT]
   → prisma.routeCycle.update({ wasCompleted: true, completedAt }) [DB: UPDATE]

3. Promise.all([
  routes.countCompletedRoutes(playerSessionId) [DB: COUNT RouteCycle]
  getNextRouteForPlayerSessionService.getNextRoute(...) (see below)
  routes.getCompletedPollinators(playerSessionId) [DB: SELECT Route.name]
])
▼
getNextRouteForPlayerSession (transaction)
  while (attempt < 300):
    prisma.$transaction(tx => {
      A. routeService.getLatestCycleNumber(tx, playerSessionId)
         → tx.routeCycle.findFirst(orderBy: cycleNumber desc) [DB: SELECT]

      B. routeService.pickRandomRouteId(tx, { excludeSeenFor: { playerSessionId, cycleNumber } })
         → buildWhere: cycles: { none: { playerSessionId, cycleNumber } }
         → tx.route.count(where) [DB: COUNT]
         → skip = crypto.randomInt(0, count)
         → tx.route.findFirst(where, orderBy: id asc, skip) [DB: SELECT]

      if picked == null (all routes seen):
        cycleNumber++

      C. tx.routeCycle.findMany(wasCompleted: true, orderBy: completedAt desc, take: 2) [DB:
SELECT]
      C. routeService.pickRandomRouteId(tx, { excludeRouteIds: [id1, id2] }) [DB: COUNT +
SELECT]
      (fallback: pickRandomRouteId with no exclusions)

      D. tx.routeCycle.create({ routeId, cycleNumber, wasCompleted: false }) [DB: INSERT]
      (throws P2002 if concurrent request got same route → retry)
    })
    LoadRouteDTOService.loadRouteDTO(routeId)
    → prisma.route.findUnique + nested nodes + facts [DB: SELECT with joins]
    ← RoutedDTO
▼
completeRoute handler (api.ts) [continued]
  newActiveRoute = ReshapeUtilities.mapRouteDTOToLegacyRoute(result.newActiveRoute)
  res.status(200).json({
    success: true,
    message: "Route completed successfully",
    newActiveRoute, ← { routeId, routeNodes[], factNodes[] }
    routesCompleted, ← integer count
    completedPollinators, ← string[] of pollinator names
  })
▼
Nginx → Player's Phone
(HTTP/1.1 200 with JSON body)

```

Portal Request (e.g. **POST /api/route-stats/total-routes-completed-by-session**)

```

Staff Browser
| POST /api/route-stats/total-routes-completed-by-session
| { "sessionId": 202401010 }
▼
Nginx :3001 (server block 2)
| location /api/
| proxy_pass http://backend:4000
▼

```



```
Express app (app.ts)
| globalLimiter, no authenticateJWT
| routeStatsTotalRoutesCompletedBySession handler
▼
handler (api.ts)
| sessionId = APIUtilities.toNumber(req.body.sessionId)
| value = await GetTotalRoutesCompleted.getroutescompleted(sessionId)
|   → prisma.routeCycle.count({ where: { sessionId, wasCompleted: true } })
| res.status(200).json({ success: true, scope: "session", scopeValue, metric, value })
▼
Staff Browser ← { "success": true, "value": 47, ... }
```
